

Atividade Objetiva 3 Resultados para Yuske Eduardo Velit Arruda

❗ As respostas corretas estão ocultas.

Pontuação desta tentativa: 15 de 15

Enviado 8 dez em 19:25

Esta tentativa levou 5 minutos.



Pergunta 1

3 / 3 pts

Você está modelando a entidade `Project` e seu repositório com TypeORM em uma aplicação NestJS. Veja os trechos de código:

```
// project.entity.ts

@Entity()
export class Project {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;

  @Column({ default: true })
  active: boolean;
}

// project.repository.ts

@Injectable()
export class ProjectRepository {
  constructor(
    @InjectRepository(Project)
    private readonly repo: Repository<Project>,
  ) {}

  async findByName(name: string): Promise<Project | null> {
    return this.repo.findOne({ where: { name } });
  }

  async save(project: Project): Promise<Project> {
    return this.repo.save(project);
  }
}
```

Analizando a implementação acima, qual alternativa descreve corretamente a composição e responsabilidades das camadas envolvidas?

- ☐ O método `findByName` viola o princípio do domínio pois mistura query com regra de negócio.
- ☒ O padrão Repository está sendo corretamente aplicado ao encapsular a lógica de persistência, permitindo que a aplicação acesse o banco sem expor a API direta do TypeORM.
- ☐ A entidade Project está acoplada à infraestrutura do banco e por isso deveria estar no mesmo arquivo do repositório.
- ☐ A entidade deveria conter validações como `@IsString()` e `@IsBoolean()` diretamente para manter a consistência da lógica.
- ☐ O repositório está incorretamente injetando o `Repository<Project>` ao invés de construir manualmente os métodos.



Pergunta 2

3 / 3 pts

Considere o seguinte trecho de código de um controller em NestJS:

```
import { Controller, Get } from '@nestjs/common';

@Controller('products')
export class ProductsController {
  @Get()
  findAll(): string {
    return 'Lista de produtos';
  }
}
```

Qual das alternativas representa corretamente o funcionamento deste trecho?

- ☒ Expõe um endpoint GET em `/products` que retorna uma string como resposta.
- ☐ Cria uma rota `/products` que responde a requisições POST com uma string fixa.
- ☐ Cria uma entidade com rota de persistência no banco de dados.
- ☐ Define um controller que responde à rota `/` com o verbo GET.



Pergunta 3

3 / 3 pts

Você está desenvolvendo a lógica de negócio de uma aplicação NestJS. O caso de uso deve buscar um produto no banco, garantir que ele esteja ativo, e em seguida devolver a resposta ao controller.

Veja parte do código abaixo:

```
@Injectable()
export class FindProductUseCase {
  constructor(private readonly repository: ProductRepository) {}

  async execute(id: number): Promise<Product> {
    const product = await this.repository.findById(id);

    if (!product || !product.active) {
      throw new NotFoundException('Produto não encontrado ou inativo');
    }

    return product;
  }
}
```

Qual afirmação está mais alinhada com os princípios de design do código acima?

- ☐ O uso de exceptions compromete a estabilidade da aplicação e deveria ser evitado.
- ☐ A injeção de dependência deveria ocorrer no controller, e não no service.
- ☐ O uso de exceptions compromete a estabilidade da aplicação e deveria ser evitado.
- ☐ A validação de negócio está acoplada à camada de infraestrutura, tornando a manutenção difícil.



O código segue boas práticas ao centralizar regras de negócio em um caso de uso, mantendo o controller mais limpo e reutilizável.



Pergunta 4

3 / 3 pts

O padrão de repositório visa abstrair o acesso à base de dados e isolar a lógica de persistência. No NestJS, é comum utilizar o `@InjectRepository()` do TypeORM para injetar o repositório padrão e construir uma camada personalizada.

Considere o exemplo a seguir:

```
// task.repository.ts
@Injectable()
export class TaskRepository {
  constructor(
    @InjectRepository(Task)
    private readonly repo: Repository<Task>,
  ) {}

  async findPendingTasks(): Promise<Task[]> {
    return this.repo.find({ where: { status: 'pending' } });
  }
}
```

```

    }

    async save(task: Task): Promise<Task> {
        return this.repo.save(task);
    }
}

```

Qual das afirmações a seguir descreve corretamente o uso do padrão repositório aplicado neste código?

- ☐ O uso de `@InjectRepository()` é inadequado, pois obriga a exposição direta do TypeORM em todas as classes.
- ☐ O repositório está acoplado ao controller, o que compromete a independência da camada de domínio.
- ☒ O código representa um repositório customizado, que encapsula o acesso ao banco e permite evolução da lógica de consulta sem impactar outras camadas.
- ☐ A chamada ao método `find()` deveria estar no controller para evitar duplicação de código.
- ☐ A injeção do repositório impede o uso de DTOs, tornando a aplicação mais rígida.



Pergunta 5

3 / 3 pts

Considere a seguinte estrutura simplificada de um caso de uso para criação de um novo projeto com validações de negócio:

```

@Injectable()
export class CreateProjectUseCase {
    constructor(private readonly repository: ProjectRepository) {}

    async execute(dto: CreateProjectDto): Promise<Project> {
        const existing = await this.repository.findByName(dto.name);

        if (existing) {
            throw new ConflictException('Projeto com este nome já existe');
        }

        return this.repository.save({
            name: dto.name,
            description: dto.description,
            createdAt: new Date()
        });
    }
}

```

Com base no código acima, qual das alternativas melhor descreve os princípios aplicados na implementação do caso de uso?

- ☐ O padrão Repository está sendo violado, pois a entidade é criada fora da camada de repositório.
- ☐ A exceção utilizada deveria ser BadRequestException pois trata-se de erro do cliente.



O DTO está sendo usado de forma indevida, pois deveria ser convertido em entidade diretamente pelo controller.



O uso de verificação de existência, criação de entidade e salvamento está concentrado em um único ponto de decisão de negócio, o que é desejado para casos de uso claros e reutilizáveis.



A lógica de persistência está misturada com a regra de negócio, o que viola o princípio de separação de responsabilidades.

Pontuação do teste: 15 de 15